

## Chapter 10

# Vector Stores

---

*How LLM applications store, index, and semantically search embedding vectors — the storage backbone of every RAG pipeline.*

---

### WHAT THIS CHAPTER COVERS

- ▶ Why keyword matching fails for semantic similarity
- ▶ Embeddings as the foundation of vector search
- ▶ Core features of a vector store
- ▶ Indexing and approximate nearest-neighbor search
- ▶ Vector Store vs Vector Database
- ▶ LangChain's common vector store interface
- ▶ Chroma DB data hierarchy
- ▶ CRUD & similarity search operations with Chroma

# Vector Stores

## RECAP — CHAPTER 9

Chapter 9 covered Text Splitters — Length-Based, Text Structure-Based (RecursiveCharacterTextSplitter), Document Structure-Based, and Semantic Meaning-Based splitting, which break large documents into chunks small enough to embed and retrieve efficiently. This chapter picks up right after chunking: once text is split into chunks, those chunks need to be converted into embedding vectors and stored somewhere that supports fast semantic search — that "somewhere" is a Vector Store.

## 1. Why Vector Stores Are Needed

### The Motivating Problem: A Movie Recommender System

Imagine building an IMDb-style website that lists every movie in the world. A user opens a movie's page, and the system should recommend similar movies at the bottom of the page. The first, simplest approach to this is *keyword matching*: compare two movies on structured attributes such as director, lead actor, genre, and release year. If most of these attributes match, the movies are considered similar.

#### 1. FLAW ONE — FALSE POSITIVES

Keyword matching can produce recommendations that are logically wrong. For example, *My Name Is Khan* and *Kabhi Alvida Naa Kehna* share the same director (Karan Johar), the same lead actor (Shah Rukh Khan), a similar release period, and an overlapping genre (Drama) — so the system rates them as highly similar. In reality, their storylines are completely different, making this a poor recommendation.

#### 2. FLAW TWO — FALSE NEGATIVES

Conversely, two movies can be thematically very similar while sharing almost no structured keywords. *Taare Zameen Par* and *A Beautiful Mind* both center on a protagonist struggling with a condition while being brilliant in another domain — a strong thematic similarity. But their directors, actors, and release years are all different, so keyword matching never identifies them as similar.

#### COMMON MISTAKE

Assuming that matching structured metadata (director, actor, genre, year) is a reliable proxy for actual content similarity. Metadata overlap does not imply that two pieces of content mean the same thing.

### The Better Approach: Compare the Actual Story

A more reliable idea is to compare the *plot* (the story) of two movies directly. If two plots are semantically similar, the movies are similar — regardless of who directed or acted in them. This requires comparing the semantic meaning of two pieces of text, which is traditionally a very difficult NLP task. Deep learning solved this with *embeddings*.

## Embeddings: Turning Text Into Comparable Vectors

*Embeddings* are a technique that passes a piece of text through a neural network, which captures its semantic meaning and represents it as a vector (a list of numbers) — commonly with 512, 784, or more dimensions. Once every movie's plot is converted into an embedding vector, similarity between movies can be computed as the angular distance (cosine similarity) between their vectors: a small angular distance means high similarity, and a large angular distance means low similarity.

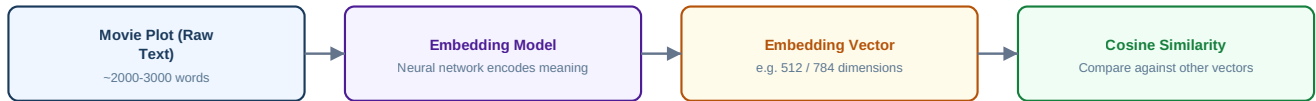


Figure 10.1 — From raw movie plot text to an embedding vector to a similarity comparison.

## Three Engineering Challenges This Creates

1. **Generating embeddings at scale** — a catalog may contain millions of movies, and an embedding vector must be generated for every single one.
2. **Storage** — embedding vectors, once generated, must be persisted so they can be reused for every future similarity search. Traditional relational databases (MySQL, Oracle) cannot efficiently store vectors or compute similarity between them — a different kind of database is needed.
3. **Fast semantic search** — given a query vector, finding the closest match among millions of stored vectors by brute-force, one-by-one comparison is computationally expensive and far too slow for a good user experience. An intelligent, faster search mechanism is required.

### IMPORTANT

All three of these challenges — generating embeddings at scale, storing them efficiently, and searching them quickly — are exactly what a Vector Store is designed to solve.

### 🔄 Quick Revision — Why Vector Stores Are Needed

- Keyword matching (director, actor, genre, year) produces both false positives and false negatives when judging content similarity.
- Comparing the actual plot/content, rather than metadata, is a more reliable similarity signal.
- Embeddings convert text into numerical vectors that capture semantic meaning.
- Similarity between two pieces of text is computed as the angular distance (cosine similarity) between their embedding vectors.
- Vector stores exist to solve three challenges: generating embeddings at scale, storing them efficiently, and searching them fast.

## 2. What Are Vector Stores?

### Definition

#### IMPORTANT

A **Vector Store** is a system designed to store and retrieve data represented as numerical vectors. Any application that needs to store embedding vectors and later retrieve them should use a vector store to handle that responsibility.

### Core Features

Every vector store provides four key features:

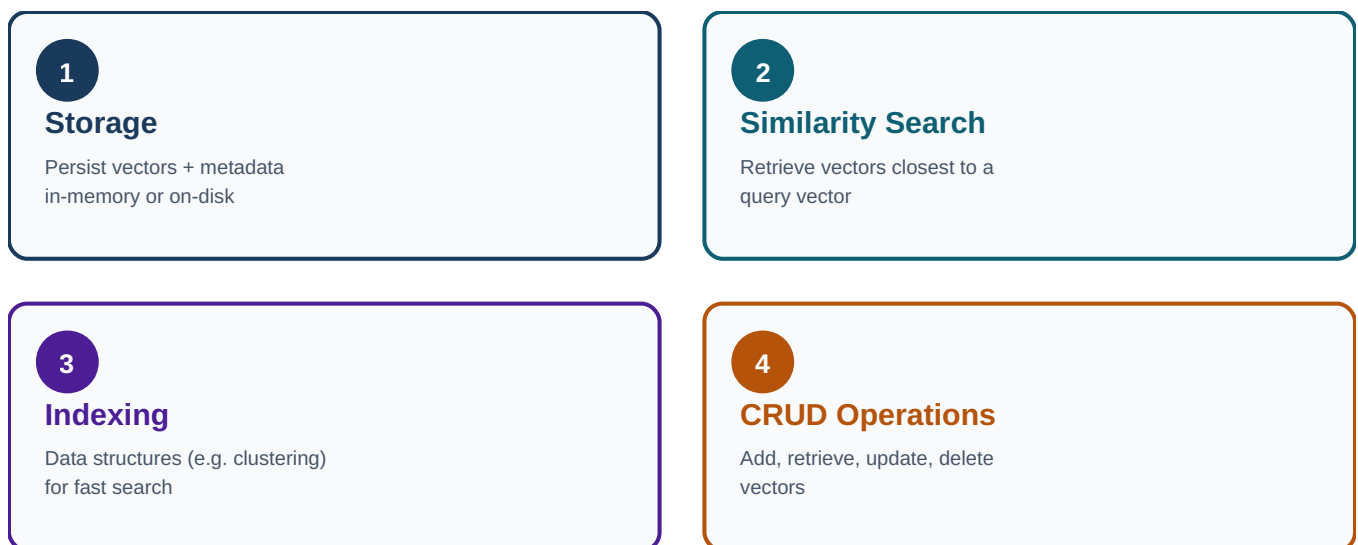


Figure 10.2 — The four core features every vector store provides.

### 1. STORAGE

Vectors and their associated metadata are retained either *in-memory* (RAM — fast, but lost when the application closes) or *on-disk* (persistent — survives an application restart). Small prototypes typically use in-memory storage; enterprise-scale applications use on-disk storage. Alongside each vector, arbitrary associated metadata (e.g. a movie's ID or name) can also be stored.

### 2. SIMILARITY SEARCH

Given a query vector, the store compares it against all stored vectors and returns the most similar ones, ranked by a similarity score.

### 3. INDEXING

A data structure and method that enables fast similarity search over high-dimensional vectors, avoiding a slow, linear  $O(n)$  comparison against every stored vector. Section 3 covers this in detail with a clustering-based example.

## 4. CRUD OPERATIONS

Like any database, a vector store supports adding new vectors, retrieving existing ones, updating them, and deleting outdated ones.

### Use Cases

- **Recommender systems** — as demonstrated with the movie recommendation example.
- **Semantic search** — any application that needs to find content with similar meaning.
- **Retrieval-Augmented Generation (RAG)** — vector stores are used extensively here, covered in depth in a later chapter.
- **Multimedia search** — searching over images or other non-text media represented as vectors.

#### REAL WORLD INSIGHT

Vector stores consistently outperform traditional relational databases for these use cases because relational databases are not built to store or efficiently retrieve high-dimensional vectors.

#### Quick Revision — What Are Vector Stores

- A vector store is a system designed to store and retrieve data represented as numerical vectors.
- Storage can be in-memory (fast, non-persistent) or on-disk (persistent, scalable).
- Similarity search retrieves the vectors most similar to a given query vector.
- Indexing provides fast similarity search over high-dimensional vectors.
- CRUD operations (add, retrieve, update, delete) are supported just like in any other database.
- Common use cases: recommender systems, semantic search, RAG, and multimedia search.

## 3. Indexing: Making Similarity Search Fast

### The Problem With Brute-Force Search

Suppose a vector store holds 10,00,000 (one million) vectors, each with 784 dimensions, and a new query vector arrives. Finding the closest stored vector by comparing the query against every single stored vector is a linear,  $O(n)$  search — 10,00,000 similarity computations. This is very slow and makes the application feel sluggish.

### How It Works: A Clustering-Based Example

Indexing solves this by avoiding a full linear scan. One simple technique uses clustering:

1. Cluster all 10,00,000 vectors into, say, 10 clusters (using any standard clustering algorithm), each holding roughly 1,00,000 vectors.
2. Compute the average (centroid) vector of each cluster, producing 10 centroid vectors.
3. When a new query vector arrives, compare it against only the 10 centroids to find the closest cluster.

4. Search only within that one closest cluster's ~1,00,000 vectors for the final best match.

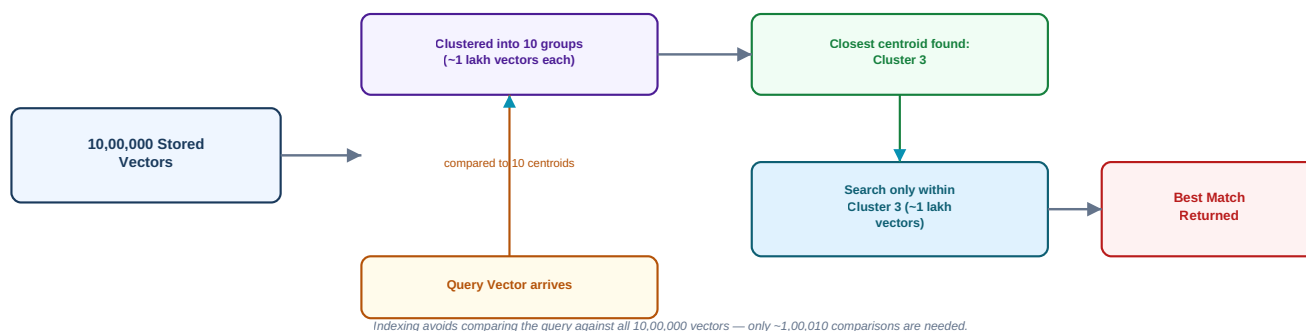


Figure 10.3 — Clustering-based indexing narrows a 1,000,000-vector search down to roughly 100,010 comparisons.

This reduces the total number of comparisons from 10,00,000 down to roughly 1,00,000 + 10, while still returning a near-optimal result. This specific technique is one example; a more widely used family of techniques for this same purpose is *Approximate Nearest Neighbor (ANN)* search, which is itself a large research topic.

### BEST PRACTICE

Understand indexing conceptually as a trade-off between search speed and exactness: indexing techniques like clustering and ANN return an approximate best match extremely fast, rather than the mathematically guaranteed best match via exhaustive search.

### Quick Revision — Indexing

- Brute-force similarity search against every stored vector is  $O(n)$  and too slow at scale.
- Indexing provides a data structure/method for fast similarity search over high-dimensional vectors.
- A simple example: cluster vectors, compare the query only against cluster centroids, then search only the closest cluster.
- Clustering-based indexing reduces comparisons from ~1,000,000 to ~100,010 in a 10-cluster setup.
- Approximate Nearest Neighbor (ANN) search is a well-known family of techniques that solves this same problem.

## 4. Vector Store vs Vector Database

The terms *Vector Store* and *Vector Database* are frequently used interchangeably online, which can be confusing. The key distinction is what additional, traditional database-style features are layered on top of the core storage + retrieval functionality.

| Aspect              | Vector Store                               | Vector Database                                                                                                    |
|---------------------|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Core capability     | Storage of vectors + similarity search     | Storage of vectors + similarity search                                                                             |
| Additional features | Typically none beyond storage & retrieval  | Distributed architecture, backup & restore, ACID transactions, concurrency control, authentication & authorization |
| Best suited for     | Prototyping and smaller-scale applications | Production environments needing significant scale                                                                  |
| Examples            | FAISS (Facebook)                           | Milvus, Qdrant, Weaviate, Pinecone                                                                                 |

### IMPORTANT

A Vector Database is effectively a Vector Store with extra database-style features. Every vector database is a vector store, but not every vector store is a vector database — a vector store does not necessarily include database-like features such as ACID transactions or role-based access control.

### INTERVIEW TIP

If asked to differentiate the two, lead with the one-line rule: "A vector database is a vector store plus traditional database features (distribution, durability, ACID transactions, authentication)." Then mention examples on each side (FAISS as a vector store; Pinecone, Milvus, Qdrant, Weaviate as vector databases).

### 🔄 Quick Revision — Vector Store vs Vector Database

- A vector store focuses on storing vectors and performing similarity search.
- A vector database adds traditional database features: distributed architecture, backup/restore, ACID transactions, concurrency control, authentication.
- Vector stores suit prototyping and smaller-scale applications; vector databases suit production-scale systems.
- Every vector database is a vector store, but not every vector store is a vector database.
- FAISS is a classic example of a lightweight vector store; Pinecone, Milvus, Qdrant, and Weaviate are vector databases.

## 5. Vector Stores in LangChain

### Why It Exists

LangChain's creators anticipated early on that LLM-based applications, especially RAG-based ones, would work heavily with embedding vectors, and therefore with vector stores and vector databases. As a result, LangChain built in support for essentially every major vector store — FAISS, Pinecone, Chroma, Qdrant, Weaviate, and more — from early on.

## How It Works: A Common Interface

LangChain wraps every vector store behind wrapper classes that share the **same method signatures**. Functions such as `from_documents/from_texts` (create a vector store), `add_documents/ add_texts` (add new vectors), `similarity_search` (perform semantic search), and metadata-based filtering all work identically regardless of which underlying vector store is being used.

### BEST PRACTICE

Because LangChain's vector store interface is consistent, an application can be built against, say, FAISS, and later swapped to Pinecone (e.g. when the application needs to scale) with minimal to no changes to the surrounding application code — only the vector store initialization changes.

### 🔄 Quick Revision — Vector Stores in LangChain

- LangChain provides built-in wrapper components for essentially every major vector store and vector database.
- All vector store wrappers share the same method signatures (a common interface).
- Core shared methods include `from_documents/from_texts`, `add_documents/add_texts`, and `similarity_search`.
- Swapping the underlying vector store (e.g. FAISS to Pinecone) requires minimal changes to application code.

## 6. Chroma DB: Hierarchy & Code Walkthrough

### What Is Chroma?

*Chroma* is a lightweight, open-source vector database, especially well-suited for local development and small-to-medium scale production needs. It sits conceptually between a plain vector store and a full-fledged vector database: it is lighter weight than heavyweight vector databases like Pinecone, yet it still offers some database-like features, giving it a flavor of both.

### Data Hierarchy

Chroma organizes data in a nested hierarchy:



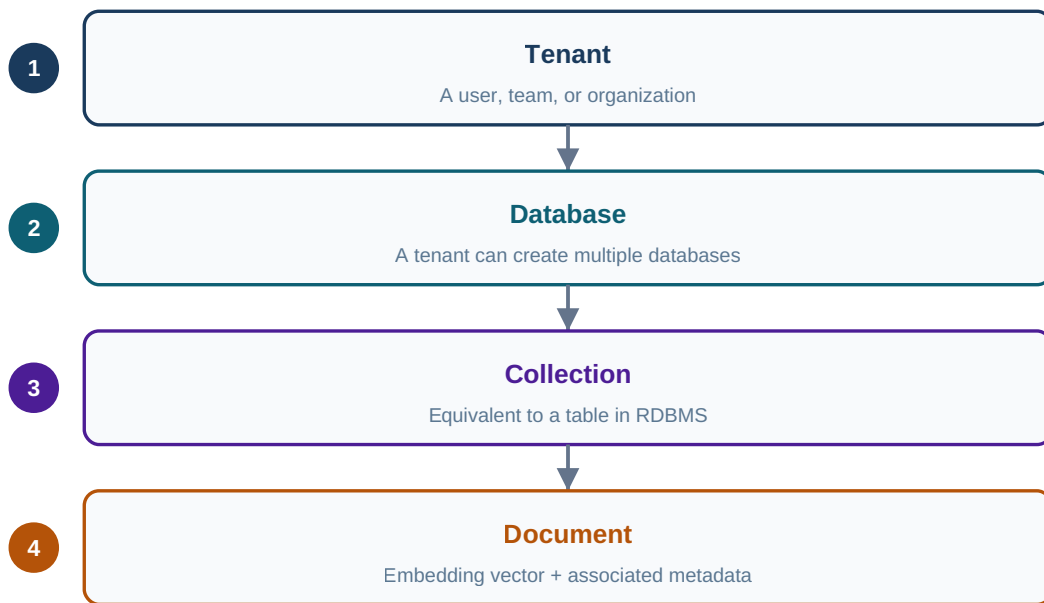


Figure 10.4 — Chroma's data hierarchy: *Tenant* → *Database* → *Collection* → *Document*.

#### REAL WORLD INSIGHT

Chroma persists its vector database as a SQLite3 file on disk. This means a Chroma `persist_directory` can technically be opened and inspected with any standard SQL client, although this is rarely necessary in practice.

## Setting Up Documents and the Vector Store

The example below creates five `Document` objects, each describing a cricket player, with the player's IPL team stored as metadata, and then initializes a Chroma vector store backed by an NVIDIA embeddings model.

```

from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings
from langchain_core.documents import Document
from langchain_chroma import Chroma
from dotenv import load_dotenv

load_dotenv()

# Embedding model used to convert page_content into vectors
embedding = NVIDIAEmbeddings(
    model = "nvidia/nemotron-3-embed-1b"
)

# Each Document holds page_content (the text) + metadata (structured info)
doc1 = Document(
    page_content = "MS Dhoni: Calm captain who led India to three ICC trophies and finished matches with powerful hitting.",
    metadata = {"team" : "Chennai Super Kings"}
)

doc2 = Document(
    page_content = "Virat Kohli: Aggressive run-machine who holds records for many centuries and dominant chasing in all formats.",
    metadata = {"team" : "Royal Challengers Bangalore"}
)

doc3 = Document(
    page_content = "Rohit Sharma: Elite opening batsman famous for scoring three double centuries in One Day Internationals and hitting massive sixes.",
    metadata = {"team" : "Mumbai Indians"}
)

doc4 = Document(
    page_content = "Yuvraj Singh: All-rounder who won the Player of the Tournament award in the 2011 World Cup and hit six sixes in one over.",
    metadata = {"team" : "Kings XI Punjab"}
)

doc5 = Document(
    page_content = "Ravindra Jadeja: World-class spin-bowling all-rounder known for his fast left-arm bowling, brilliant fielding, and clutch batting.",
    metadata = {"team" : "Rajasthan Royals"}
)

docs = [doc1, doc2, doc3, doc4, doc5]

# Creates (or connects to) a Chroma collection named "sample",
# persisted on disk inside the "chroma_db" folder
vector_store = Chroma(
    embedding_function = embedding,
    persist_directory = "chroma_db",

```

```
collection_name = "sample"
)
```

Three things are configured when constructing the `Chroma` object: the `embedding_function` (which model converts documents into vectors), the `persist_directory` (where the vectors are stored on-disk), and the `collection_name` (the collection this data belongs to, analogous to a table).

## Adding Documents

`add_documents()` inserts one or more `Document` objects into the vector store in a single call. Each document is automatically assigned a unique ID on insertion (custom IDs can also be supplied), which can later be used to retrieve, update, or delete that specific document.

```
vector_store.add_documents(docs)
# Each of the 5 documents gets embedded and stored, and returns a list of auto-generated document IDs
```

## Viewing Stored Documents

The `get()` method returns the contents of the vector store. The `include` parameter controls which fields are returned — embeddings, documents (the raw text), and/or metadatas.

```
vector_store.get(include = ["embeddings", "documents", "metadatas"])
# Returns all 5 document IDs, their embedding vectors, their raw text, and their metadata
```

## Similarity Search

`similarity_search()` performs semantic search over the stored vectors given a natural-language query. The `k` parameter controls how many top results are returned.

```
bowler = vector_store.similarity_search(
    query = "Who among these are bowlers?",
    k = 2
)
# Returns the 2 documents whose embeddings are most semantically close to the query
# (e.g. Jasprit Bumrah and Ravindra Jadeja style all-rounder/bowler profiles)
```

If a similarity score is also needed alongside the matched documents, `similarity_search_with_score()` returns both.

```
bowler = vector_store.similarity_search_with_score(
    query = "Who among these are bowlers?",
    k = 2
)
# A lower score represents a smaller distance -> a higher (better) similarity
```

### IMPORTANT

In `similarity_search_with_score()`, the returned score represents distance, not similarity itself: a **lower** score means the vectors are closer together and therefore more similar.

## Metadata-Based Filtering

Search results can also be restricted using the `filter` parameter, which matches against a document's metadata rather than (or alongside) its content.

```
csk = vector_store.similarity_search_with_score(
    query = "Move as per the Filter",
    filter = {"team" : "Chennai Super Kings"},
    k = 1
)
# Restricts the search to only documents whose metadata "team" equals "Chennai Super Kings"
```

### BEST PRACTICE

Metadata filtering is especially useful in production RAG systems — for example, restricting retrieval to documents belonging to a specific user, tenant, department, or date range before running semantic search.

## Updating an Existing Document

`update_document()` replaces the content (and re-embeds it) for an existing document, identified by its `document_id`.

```
# A revised description for the existing Virat Kohli document
updated_document = Document(
    page_content = "Virat Kohli is a master of modern cricket whose smooth strokeplay and steady run-scoring have rewritten the record books.",
    metadata = {"team" : "Royal Challengers Bangalore"}
)

vector_store.update_document(
    document_id = "0ba0f0aa-86a0-4d8d-b69a-c7531b95bd10",
    document = updated_document
)
# Re-embeds and overwrites the existing document at this ID with the new content
```

## Deleting a Document

`delete()` removes one or more documents by their IDs.

```
vector_store.delete(ids = ["0ba0f0aa-86a0-4d8d-b69a-c7531b95bd10"])
# Permanently removes the Virat Kohli document from the "sample" collection
```

## INTERVIEW TIP

Be ready to name the full CRUD + search surface Chroma exposes through LangChain: `add_documents`, `get`, `similarity_search`, `similarity_search_with_score`, `update_document`, and `delete` — and note that this exact same set of method names works identically for other LangChain vector store wrappers such as FAISS or Pinecone.

### 🔄 Quick Revision — Chroma DB

- Chroma is a lightweight, open-source vector database, well-suited to local development and small/medium production use.
- Chroma's hierarchy is Tenant -> Database -> Collection -> Document; a Document holds an embedding vector + metadata.
- Chroma persists data on-disk as a SQLite3 file when a `persist_directory` is provided.
- `add_documents()` inserts documents and auto-assigns unique IDs; these IDs are needed for `update_document()` and `delete()`.
- `get(include=[...])` inspects stored embeddings, documents, and metadata.
- `similarity_search()` and `similarity_search_with_score()` perform semantic search; a lower score means higher similarity.
- The `filter` parameter restricts similarity search using document metadata.

## K. Key Takeaways

- **Keyword matching fails** at judging true content similarity, producing both false positives (unrelated movies sharing metadata) and false negatives (similar movies sharing no metadata).
- **Embeddings** convert text into numerical vectors capturing semantic meaning, enabling similarity to be computed via cosine/angular distance.
- **Vector Stores** exist to solve three challenges: generating embeddings at scale, storing them efficiently, and searching them fast.
- **Four core features** of a vector store: Storage, Similarity Search, Indexing, and CRUD operations.
- **Indexing** (e.g. clustering-based, or Approximate Nearest Neighbor) avoids slow  $O(n)$  brute-force search by narrowing the search space intelligently.
- **Vector Database = Vector Store + database features** such as distributed architecture, ACID transactions, backup/restore, and authentication.
- **LangChain** wraps every major vector store behind a common interface, so swapping vector stores requires minimal code changes.
- **Chroma** is a lightweight vector database organized as Tenant → Database → Collection → Document, supporting add, get, similarity search (with/without score), metadata filtering, update, and delete.

## R. Revision Sheet

## Vector Store

A system designed to store and retrieve data represented as numerical vectors, offering storage, similarity search, indexing, and CRUD operations.

## Embedding

A numerical vector representation of text (or other data) that captures its semantic meaning, generated by a neural network.

## Cosine Similarity

A measure of angular distance between two vectors, used to determine how semantically similar two pieces of content are.

## Indexing

A data structure/technique (e.g. clustering, ANN) that enables fast similarity search over large collections of high-dimensional vectors, avoiding brute-force  $O(n)$  comparison.

## Vector Store vs Vector Database

A vector database is a vector store plus additional database-style features: distributed architecture, backup/restore, ACID transactions, concurrency control, and authentication.

## LangChain's Common Interface

All LangChain vector store wrappers (FAISS, Pinecone, Chroma, Qdrant, Weaviate, etc.) share the same method signatures, making the underlying store swappable with minimal code change.

## Chroma DB

A lightweight, open-source vector database organized as Tenant → Database → Collection → Document, persisted on-disk as SQLite3.

## Metadata Filtering

Restricting similarity search results using a document's metadata (e.g. `filter={"team": "Chennai Super Kings"}`) in addition to, or instead of, the query text.

## I. Interview Questions

- Q1.** Why does keyword-based matching fail to reliably identify similar content, such as similar movies?
- Q2.** What is an embedding, and how does it enable semantic similarity comparison between two pieces of text?
- Q3.** What are the three main engineering challenges that arise when building a semantic similarity system at scale?
- Q4.** What are the four core features every vector store provides?
- Q5.** Why can't traditional relational databases like MySQL or Oracle be used to store and search embedding vectors efficiently?
- Q6.** Explain how clustering-based indexing reduces the number of comparisons needed for a similarity search.
- Q7.** What is the difference between a Vector Store and a Vector Database? Give an example of each.
- Q8.** How does LangChain ensure that switching from one vector store (e.g. FAISS) to another (e.g. Pinecone) requires minimal code changes?
- Q9.** Describe Chroma DB's internal data hierarchy, from Tenant down to Document.
- Q10.** In `similarity_search_with_score()`, does a lower score mean higher or lower similarity? Explain why.

## F. Further Reading

- [LangChain Documentation — Vector Stores](#) (conceptual guide and the common vector store interface)
- [LangChain Documentation — Chroma integration](#)
- [Chroma DB official documentation](#) (collections, persistence, embedding functions)
- [FAISS \(Facebook AI Similarity Search\) — library documentation and paper](#)
- [Approximate Nearest Neighbor \(ANN\) search](#) — algorithms such as HNSW and IVF, for a deeper dive into production-grade indexing